

EXPERIMENT-1
SACHIN SINGH
24UADS1045

Objective :-

WAP to visualize the Perceptron Learning Algorithm using numpy and matplotlib/seaborn in Python. Evaluate performance of a single perceptron for NAND and XOR truth tables as input dataset.

Description of the model :-

The model used in this experiment is a single-layer Perceptron, which is the simplest form of an artificial neural network. It consists of an input layer and an output neuron, without any hidden layers.

Model Architecture

- **Input** **Layer:**
Two input neurons corresponding to the binary inputs X_1 and X_2 .
- **Weights:**
Each input is associated with a weight (w_1, w_2) that determines its contribution to the output.
- **Bias:**
A bias term b is added to shift the decision boundary.
- **Activation** **Function:**
A binary step function is used to produce the final output (0 or 1).

Mathematical Representation

The perceptron computes a weighted sum of inputs:

$$z = w_1x_1 + w_2x_2 + b$$

The output is obtained using the step activation function:

$$y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

Learning Algorithm :-

The model is trained using the Perceptron Learning Algorithm, which updates weights and bias based on the error between the predicted output and the target output.

$$\text{Error} = \text{Target} - \text{Predicted}$$

$$w_i = w_i + \eta \times \text{Error} \times x_i$$

$$b = b + \eta \times \text{Error}$$

where η is the learning rate.

Source Code :-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([1, 1, 1, 0])

np.random.seed(7)

weights = np.random.uniform(-1, 1, size=2).astype(float)
bias = -0.5 # NAND-friendly bias

learning_rate = 0.2
max_epochs = 20

def step_function(z):
    return 1 if z >= 0 else 0

def predict(x):
    return step_function(np.dot(x, weights) + bias)

accuracy_history = []
weight_history = []
bias_history = []

epoch = 0

while epoch < max_epochs:
    epoch += 1
    print(f"\n===== EPOCH {epoch} =====\n")

    epoch_data = []
    correct = 0

    for xi, target in zip(X, y):
        prediction = predict(xi)
        error = target - prediction
```

```

# Update rule
weights[:] = weights + learning_rate * error * xi
bias += learning_rate * error

if prediction == target:
    correct += 1

epoch_data.append([xi[0], xi[1], prediction, target, error])

accuracy = correct / len(y)
accuracy_history.append(accuracy)
weight_history.append(weights.copy())
bias_history.append(bias)

df = pd.DataFrame(
    epoch_data,
    columns=["X1", "X2", "Predicted", "Target", "Error"]
)
print(df)
print(f"\nAccuracy = {accuracy*100:.2f}%")

if accuracy == 1.0:
    print("\n 100% Accuracy Achieved ")
    break

time.sleep(1)

print("\nFinal Weights & Bias:")
print(f"w1 = {weights[0]:.4f}")
print(f"w2 = {weights[1]:.4f}")
print(f"bias = {bias:.4f}")

plt.figure(figsize=(6,4))
plt.plot(range(1, len(accuracy_history)+1),
         accuracy_history, marker='o')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title(" Accuracy vs Epoch (NAND)")
plt.grid(True)
plt.show()

weight_history = np.array(weight_history)
bias_history = np.array(bias_history)

plt.figure(figsize=(7,5))
plt.plot(weight_history[:,0], label="w1")

```

```

plt.plot(weight_history[:,1], label="w2")
plt.plot(bias_history, label="bias", linestyle="--")

plt.xlabel("Epoch")
plt.ylabel("Value")
plt.title("Weight & Bias Evolution")
plt.legend()
plt.grid(True)
plt.show()

x1 = np.linspace(-0.5, 1.5, 100)
x2 = np.linspace(-0.5, 1.5, 100)
xx1, xx2 = np.meshgrid(x1, x2)

Z = np.dot(np.c_[xx1.ravel(), xx2.ravel()], weights) + bias
Z = Z.reshape(xx1.shape)

plt.figure(figsize=(6,6))
plt.contourf(xx1, xx2, Z >= 0, alpha=0.3, cmap='coolwarm')

sns.scatterplot(
    x=X[:,0], y=X[:,1],
    hue=y, s=120,
    palette="Set1", edgecolor="k"
)

plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Decision Boundary – NAND")
plt.show()

# For XOR

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0]) # XOR output

# Safe initialization (avoids 0% accuracy)
weights = np.array([1.0, 1.0], dtype=float)
bias = -0.5

learning_rate = 0.2

```

```

epochs = 10

def step_function(z):
    return 1 if z >= 0 else 0

def predict(x):
    return step_function(np.dot(x, weights) + bias)

accuracy_history = []
weight_history = []
bias_history = []

for epoch in range(1, epochs + 1):
    print(f"\n===== EPOCH {epoch} =====\n")

    epoch_data = []
    correct = 0

    for xi, target in zip(X, y):
        prediction = predict(xi)
        error = target - prediction

        # Perceptron learning rule
        weights[:] = weights + learning_rate * error * xi
        bias += learning_rate * error

        if prediction == target:
            correct += 1

    epoch_data.append([xi[0], xi[1], prediction, target, error])

    accuracy = correct / len(y)
    accuracy_history.append(accuracy)
    weight_history.append(weights.copy())
    bias_history.append(bias)

    df = pd.DataFrame(
        epoch_data,
        columns=["X1", "X2", "Predicted", "Target", "Error"]
    )

    print(df)
    print(f"\nAccuracy = {accuracy*100:.2f}%")

    time.sleep(1)

print("\nFinal Learned Parameters:")

```

```

print(f"Weight w1 = {weights[0]:.4f}")
print(f"Weight w2 = {weights[1]:.4f}")
print(f"Bias    = {bias:.4f}")

plt.figure(figsize=(6,4))
plt.plot(range(1, len(accuracy_history) + 1),
         accuracy_history, marker='o')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Epoch (XOR – Single Perceptron)")
plt.grid(True)
plt.show()

weight_history = np.array(weight_history)
bias_history = np.array(bias_history)

plt.figure(figsize=(8,5))
plt.plot(weight_history[:, 0], marker='o', label="Weight w1")
plt.plot(weight_history[:, 1], marker='s', label="Weight w2")
plt.plot(bias_history, marker='^', linestyle='--', label="Bias")

plt.xlabel("Epoch")
plt.ylabel("Parameter Value")
plt.title("Weight & Bias Evolution for XOR (Single-Layer Perceptron)")
plt.legend()
plt.grid(True)
plt.show()

x1_vals = np.linspace(-0.5, 1.5, 100)
x2_vals = np.linspace(-0.5, 1.5, 100)
xx1, xx2 = np.meshgrid(x1_vals, x2_vals)

Z = np.dot(np.c_[xx1.ravel(), xx2.ravel()], weights) + bias
Z = Z.reshape(xx1.shape)

plt.figure(figsize=(6,6))
plt.contourf(xx1, xx2, Z >= 0, alpha=0.3, cmap='coolwarm')

sns.scatterplot(
    x=X[:,0], y=X[:,1],
    hue=y, s=120,
    palette="Set1", edgecolor='k'
)

plt.xlabel("Input X1")
plt.ylabel("Input X2")
plt.title("Decision Boundary for XOR (Single Perceptron)")

```

plt.show()

Output for NAND :-

===== EPOCH 1 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	1	1
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	0	0	0

Accuracy = 50.00%

===== EPOCH 2 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	1	1
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 25.00%

===== EPOCH 3 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 4 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 5 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	0	0	0

Accuracy = 75.00%

===== EPOCH 6 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 7 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 8 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	0	1	1
2	1	0	1	1	0
3	1	1	0	0	0

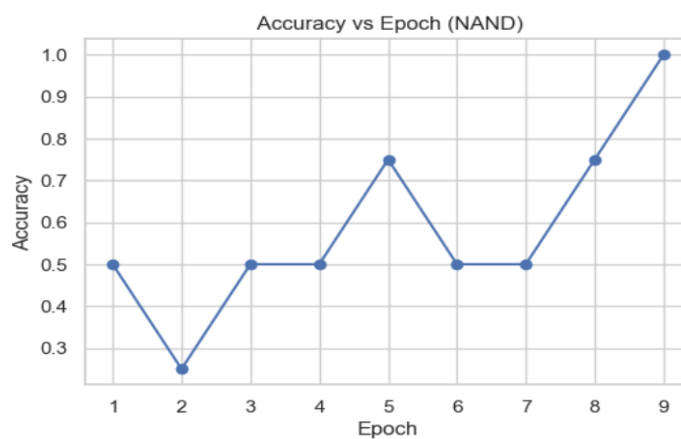
Accuracy = 75.00%

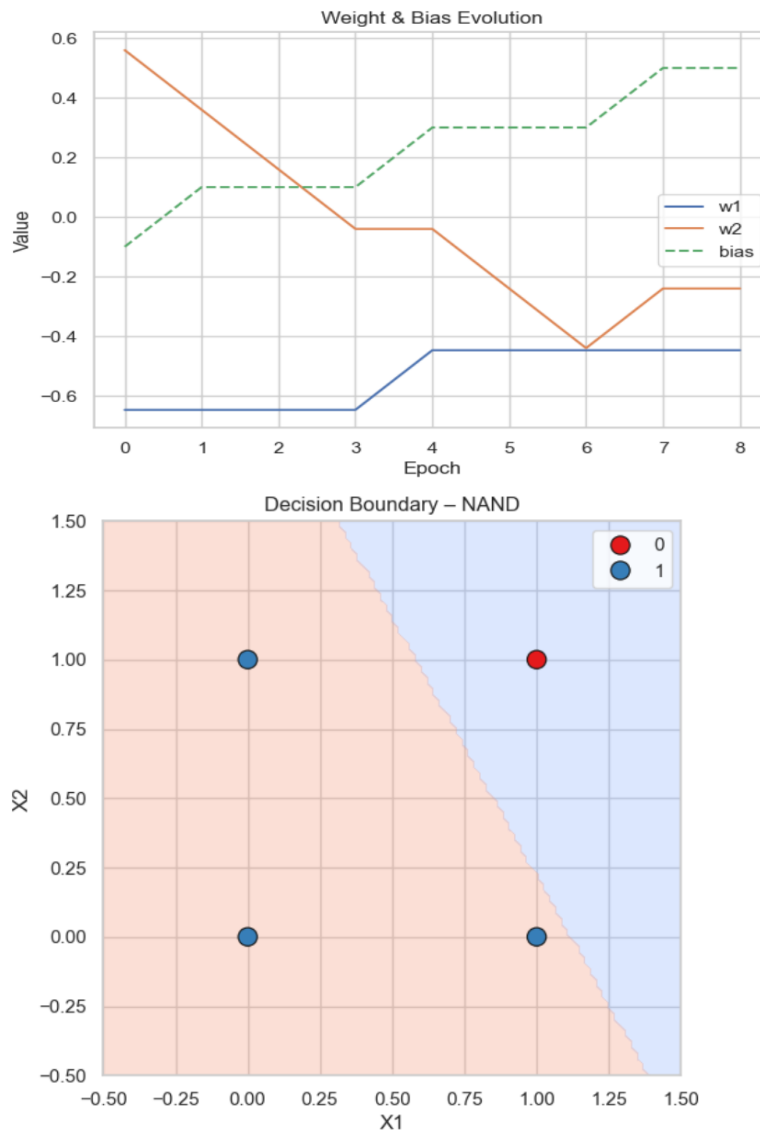
===== EPOCH 9 =====

	X1	X2	Predicted	Target	Error
0	0	0	1	1	0
1	0	1	1	1	0
2	1	0	1	1	0
3	1	1	0	0	0

Accuracy = 100.00%

100% Accuracy Achieved





Output for XOR :-

===== EPOCH 1 =====

X1	X2	Predicted	Target	Error
0	0	0	0	0
1	0	1	1	0
2	1	1	1	0
3	1	1	0	-1

Accuracy = 75.00%

===== EPOCH 2 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	0	0
1	0	1	1	1	0
2	1	0	1	1	0
3	1	1	1	0	-1

Accuracy = 75.00%

===== EPOCH 3 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	0	0
1	0	1	0	1	1
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 25.00%

===== EPOCH 4 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	0	0
1	0	1	0	1	1
2	1	0	1	1	0
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 5 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	0	0
1	0	1	0	1	1
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 25.00%

===== EPOCH 6 =====

	X1	X2	Predicted	Target	Error
0	0	0	0	0	0
1	0	1	1	1	0
2	1	0	0	1	1
3	1	1	1	0	-1

Accuracy = 50.00%

===== EPOCH 7 =====

	X1	X2	Predicted	Target	Error
--	----	----	-----------	--------	-------

0	0	0	0	0	0
---	---	---	---	---	---

1	0	1	0	1	1
---	---	---	---	---	---

2	1	0	1	1	0
---	---	---	---	---	---

3	1	1	1	0	-1
---	---	---	---	---	----

Accuracy = 50.00%

===== EPOCH 8 =====

	X1	X2	Predicted	Target	Error
--	----	----	-----------	--------	-------

0	0	0	0	0	0
---	---	---	---	---	---

1	0	1	0	1	1
---	---	---	---	---	---

2	1	0	0	1	1
---	---	---	---	---	---

3	1	1	1	0	-1
---	---	---	---	---	----

Accuracy = 25.00%

===== EPOCH 9 =====

	X1	X2	Predicted	Target	Error
--	----	----	-----------	--------	-------

0	0	0	0	0	0
---	---	---	---	---	---

1	0	1	1	1	0
---	---	---	---	---	---

2	1	0	0	1	1
---	---	---	---	---	---

3	1	1	1	0	-1
---	---	---	---	---	----

Accuracy = 50.00%

===== EPOCH 10 =====

	X1	X2	Predicted	Target	Error
--	----	----	-----------	--------	-------

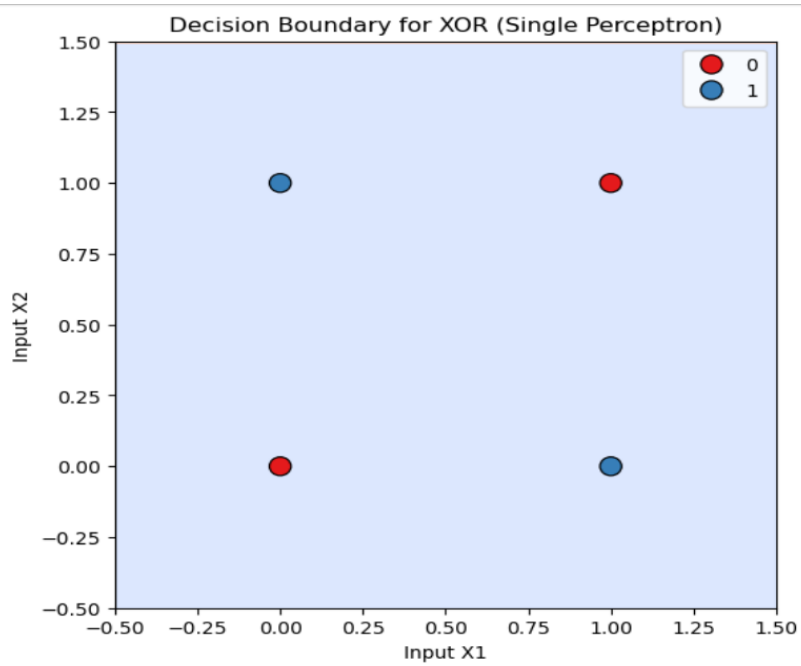
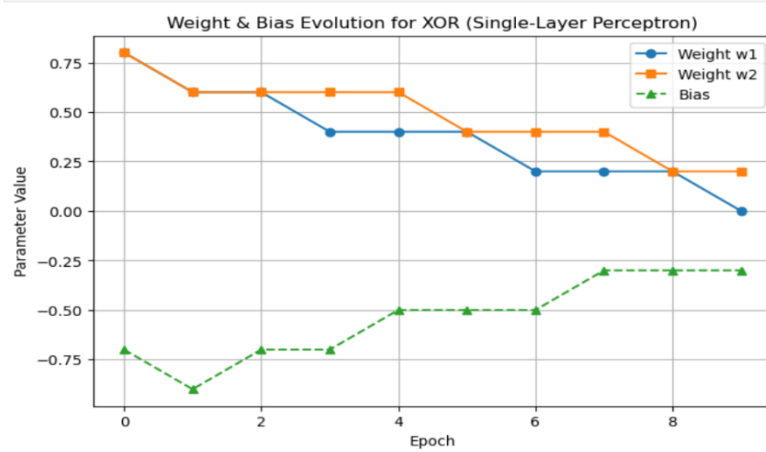
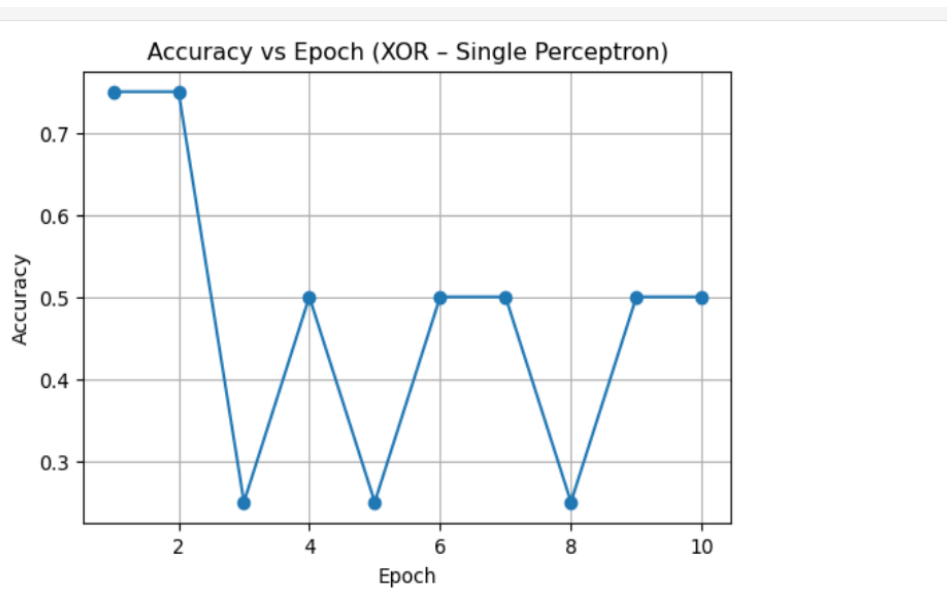
0	0	0	0	0	0
---	---	---	---	---	---

1	0	1	0	1	1
---	---	---	---	---	---

2	1	0	1	1	0
---	---	---	---	---	---

3	1	1	1	0	-1
---	---	---	---	---	----

Accuracy = 50.00%



Description of the Code :-

1. Importing Required Libraries

- NumPy: Used for numerical operations such as dot products and array manipulation.
- Pandas: Used to display epoch-wise results in a tabular form.
- Matplotlib & Seaborn: Used to plot accuracy graphs, weight evolution graphs, and decision boundaries.
- Time: Used to introduce a 1-second delay after each epoch to observe learning step-by-step.

2. Dataset Definition

Two different datasets are used:

NAND Dataset

- Inputs represent the NAND truth table.
- Output is 1 for all cases except when both inputs are 1.
- NAND is a **linearly separable** problem.

XOR Dataset

- Inputs represent the XOR truth table.
- Output is 1 only when inputs are different.
- XOR is not linearly separable.

The datasets are stored as NumPy arrays for efficient computation.

3. Initialization of Parameters

- Weights are initialized either randomly or with safe fixed values.
- A bias term is included to shift the decision boundary.
- Learning rate controls the speed of weight updates.
- Number of epochs determines how many times the dataset is iterated.

4. Activation Function

- A binary step activation function is used.
- It outputs:
 - 1 if the weighted sum is greater than or equal to zero
 - 0 otherwise

5. Prediction Function

- Computes the weighted sum of inputs and bias.
- Applies the step activation function.
- Produces binary output (0 or 1).

6. Training Loop (Perceptron Learning Algorithm)

- The model is trained for multiple epochs.
- For each input pattern:

- Output is predicted.
 - Error is calculated as the difference between target and predicted output.
 - Weights and bias are updated using the perceptron learning rule.
- Accuracy is calculated after each epoch.

7. Epoch-wise Output Table

- For every epoch, a Pandas DataFrame displays:
 - Input values
 - Predicted output
 - Target output
 - Error
- Accuracy after each epoch is printed.

8. Performance Tracking

- Accuracy values are stored to analyze learning performance.
- Weight and bias values are stored after each epoch to visualize learning behavior.

9. Graphical Visualizations

The following plots are generated for both NAND and XOR:

1. Accuracy vs Epoch Graph
2. Weight and Bias Evolution Graph
3. Decision Boundary Plot

These visualizations help in understanding convergence and limitations.

10. Interpretation of Results

NAND Gate

- Accuracy reaches **100%**.
- Weights and bias converge.
- Decision boundary perfectly separates the classes.
- Confirms that NAND is linearly separable.

XOR Gate

- Accuracy remains between **50% and 75%**.
- Weights and bias oscillate.
- Decision boundary fails to separate classes completely.
- Confirms that XOR is not linearly separable.

Performance Evaluation (Accuracy) :-

The performance of the perceptron model is evaluated using classification accuracy, defined as the ratio of correctly classified input patterns to the total number of patterns.

For the NAND gate, the perceptron achieves **100%** accuracy within a few epochs. The accuracy stabilizes at 1.0, indicating successful convergence due to the linear separability of the NAND function

For the XOR gate, the perceptron accuracy fluctuates between **50% and 75%** across epochs. The model does not converge to 100% accuracy because XOR is not linearly separable. Increasing the number of epochs does not improve accuracy further.

MY COMMENTS :-

It was the first experiment so it gave me idea about fundamental of neural network that it can be build starting from this simple algorithm

In this particular experiment the things I learned are as follows :

- 1 Starting with truth table of the NAND and OR Gate I learned how to use them as input and target points
2. When I was experimenting with learning rate the less the learning rate more the epochs were required for better accuracy.
3. Learned how weights are updated on each epoch and how accuracy of each epoch is fluctuating from 25% to 75%
4. For NAND gate the algorithm showed convergence means that it is linearly seperable using a decision boundary
5. But for XOR gate in the beginning while experimenting values of hyperparameters it was not converging after few epochs and giving 0 accuracy, that shows that it is not linearly seperable but after some alterations it was possible to achieve 50% and accuracy.
6. How to see the plots for this experiment and know about the changing decision boundary, like by changing bias the line parallelly shifts and by changing the weights it changes angle of the decision boundary.